

CPT204 Data Structures and Algorithms - 24-25R

Part A. Gap-fill Questions

1. In the following code, 'number' is a __ variable as it stores the actual values of primitive data types directly, while 'message' is a __ variable as it stores the address of an object in memory.

```
public class Example {  
    public static void main(String[] args) {  
        int number = 42;  
        String message = "Hello, world!";  
    }  
}
```

sol:

primitive, reference

2. A __ is a class from which other classes are derived, while a __ is a class that is derived from another class.

sol:

superclass, subclass

3. In Java, __ equality checks if two variables refer to the exact same object in memory, while __ equality checks if two objects have the same state or content, as defined by the equals() method.

sol:

reference, value

4. An __ can have instance methods that implement a default behavior, while an __ cannot have instance methods.

sol:

abstract class, interface

5. For scenarios requiring efficient random access through an index without frequent insertion or removal of elements except at the end of the list, the __ class is more suitable. In contrast, for applications that frequently insert or delete elements at the beginning of the list, the __ class is a better choice.

sol:

ArrayList LinkedList

6. In the program below, a compilation error occurs at line 4 because the Fruit class does not implement the __ interface , which is required for the Arrays.sort() method to sort the array of Fruit objects.

```
public class Test {
    public static void main(String[] args) {
        Frust[] fruits = {new Fruit(2), new Fruit(1), new Fruit(3)};
        Arrays.sort(fruits);
    }
}

class Fruit {
    private double weight;
    public Fruit(double weight) {
        this.weight = weight;
    }
}
```

sol:

Comparable

7. The following statement would encounter a compilation error because a generic type must be a type. One way to correct the statement is to change the type parameter “int” to __

```
ArrayList<int> intList = new ArrayList<>();
```

sol:

Integer

8.

The method header is left blank in the following code. Fill in the header using a generic method __;

```
public class GenericMethodDemo {
    public static void main(String[] args) {
        Integer[] numbers = {10, 20, 30, 40, 50};
        String[] cities = {"Tokyo", "London", "Paris", "New York"};
        print(numbers);
        print(cities);
    }
}
```

```

public static ____ {
    for (int i = 0; i < list.length; i++) {
        System.out.print(list[i] + " ");
        System.out.println();
    }
}
}

```

sol

```
<E> void print(E[] list)
```

9. In the code below, 'LinkedList' is a __ without specifying any type parameter, which implies it can hold elements of any type.

```

public class Example {
    public static void main(String[] args) {
        LinkedList list = new LinkedList();
        list.add("World");
        list.add(24);
        list.add(2.71);
    }
}

```

sol:

raw type raw type are like this: `LinkedList list = new LinkedList();` in the compiling period, the compiler do not know what the element's type. you have to make sure the type safety yourself; Parameterized type are like this: `LinkedList<String> list = new LinkedList<>();` the compiler can checks the type safety for you, if you try to add an element that is not a String, the compiler will throw an error. Wildcard type: `LinkedList<?> list = new LinkedList<>();` This is used in read only situations. the container only return Object type. Producer Extends, Consumer Super for write situations: `LinkedList<? extends T> list; List<? super Integer> list = new ArrayList<Number>(); list.add(10);` Yes 可以添加 Integer `list.add(20);` Yes Object `obj = list.get(0);` 读出来只能是 Object

for read situations: `LinkedList<? super T> list; List<? extends Number> list = new ArrayList<Integer>(); list.add(10);` No 编译错误——不知道具体是 Integer 还是 Double `Number n = list.get(0);` Yes 读出来至少是 Number

10. A list collection organizes elements in a __ order, and permits specifying the position where an element is stored. Additionally, elements can be accessed by __

sol:

insertion (sequential) index

11. The `__` interface defines the natural ordering of objects of each class that implements it, whereas the `__` interface defines a separate class to compare objects of another class.

NOTE: TO BE REVIEW

sol:

Comparable Conparator

12. The `__` method in the Queue interface retrieves and removes the head of this queue, or null if this queue is empty. The `__` method in the Queue interface retrieves and removes the head of this queue and throws an exception if this queue is empty.

note:

Queue 自身方法

方法	返回值	解释
<code>add(E e)</code>	boolean	将元素插入队尾，容量不足时抛出 <code>IllegalStateException</code>
<code>offer(E e)</code>	boolean	将元素插入队尾，容量不足时返回 false（优先使用）
<code>remove()</code>	E	移除并返回队头元素，队列为空抛出 <code>NoSuchElementException</code>
<code>poll()</code>	E	移除并返回队头元素，队列为空返回 null
<code>element()</code>	E	查看队头元素不移除，队列为空抛出 <code>NoSuchElementException</code>
<code>peek()</code>	E	查看队头元素不移除，队列为空返回 null

继承自 `Collection<E>`

方法	返回值	解释
<code>size()</code>	int	返回队列中元素个数
<code>isEmpty()</code>	boolean	队列为空返回 true
<code>contains(Object o)</code>	boolean	是否包含某个元素（ <code>equals()</code> 比较）
<code>iterator()</code>	<code>Iterator<E></code>	返回按元素顺序遍历的迭代器
<code>toArray()</code>	<code>Object[]</code>	返回包含所有元素的 <code>Object[]</code> 数组
<code>toArray(T[] a)</code>	<code>T[]</code>	返回指定类型 <code>T[]</code> 的数组
<code>remove(Object o)</code>	boolean	移除第一个匹配元素，成功返回 true
<code>containsAll(Collection<?> c)</code>	boolean	是否包含参数集合的所有元素
<code>addAll(Collection<? extends E> c)</code>	boolean	将参数集合所有元素追加到队尾
<code>removeAll(Collection<?> c)</code>	boolean	移除与参数集合共有的元素（差集）
<code>retainAll(Collection<?> c)</code>	boolean	只保留与参数集合共有的元素（交集）
<code>clear()</code>	void	清空所有元素
<code>equals(Object o)</code>	boolean	比较两个集合是否相等
<code>hashCode()</code>	int	返回队列哈希值

Java 8 新增

方法	返回值	解释
stream()	Stream<E>	返回串行流
parallelStream()	Stream<E>	返回并行流
splitterator()	Spliterator<E>	返回用于遍历分割的 Spliterator
removeIf(Predicate<? super E> filter)	boolean	移除所有满足条件的元素
forEach(Consumer<? super E> a	void	对每个元素执行操作

sol:

poll() remove()

13. Suppose list list1 is [1, 3, 5] and list list2 is [2, 4, 6]. After list1.addAll(list2), list1 is __

sol:

[1, 3, 5, 2, 4, 6]

14. In the following code, the key “101” will correspond to the value ” __” after all the operations.

```
public class Test {
    public static void main(String[] args) {
        Map<String, String> map = new HashMap<>();
        map.put("101", "Alice");
        map.put("102", "Bob");
        map.put("101", "Charlie");
    }
}
```

sol:

” Charlie”

15. The output of the following code is __

```
HashMap<String> map = new HashMap<>();
map.put("One", 1);
map.put("Two", 2);
map.put("Three", 3);
System.out.println(map.get("Two"));
```

sol:

2

16. In the selection sort algorithm, the __ element in the array is identified and exchanged with the __ element in each iteration.

sol:

smallest first unsorted

17. An algorithm with the $O(n)$ time complexity is called a __ algorithm and an algorithm with the $O(\log n)$ time complexity is called a __ algorithm.

NOTE: TO BE REVIEWED

sol:

linear Logarithmic

18. __ is a method for solving complex problems by breaking them down into simpler subproblems where these subproblems overlap, unlike in __ method where subproblems are more independent.

sol:

Dynamic Programming / Divide and Conquer DP 拆解的子问题相互重叠 (用 memoization/table 避免重复计算), 分治法的子问题相互独立。

19. Use the Big O notation to estimate the time complexity of the following method:

```
public static void mD(int[] m) {
    for (int i = 0; i < m.length; i++) {for (int j = 0; j < m.length; j++) {
        System.out.print(m[i] + " ");
        for (int j= 0; i < m.length; i++) {
            System.out.print(m[i] + " ");
        }
    }
}
```

sol:

$O(n^3)$

20. Suppose a list is [12, 9, 15, 4]. After the second pass of bubble sort, the list becomes .

sol:

- 12 vs 9 → 交换 → [9, 12, 15, 4]
- 12 vs 15 → 不动 → [9, 12, 15, 4]
- 15 vs 4 → 交换 → [9, 12, 4, 15]

第 2 轮:

- 9 vs 12 → 不动 → [9, 12, 4, 15]
- 12 vs 4 → 交换 → [9, 4, 12, 15]
- 12 vs 15 → 不动 → [9, 4, 12, 15]

21. What value should best replace the ‘?’ to maintain the max heap strictly?

sol:

11

22. A __ graph is a simple undirected graph in which every pair of distinct vertices is connected by a unique edge, while in a __ graph, not every pair of vertices is connected by an edge.

sol:

complete / sparse (或 complete / incomplete)

- Complete graph (完全图): 每对不同顶点之间都有一条唯一边连接。
- Sparse/incomplete graph (稀疏图/非完全图): 不是所有顶点对之间都有边。

23. is a graph traversal method that visits all the vertices of a graph as deeply as possible before backtracking, while __ graph level by level.

sol:

DFS BFS

24. __ is an algorithm in graph theory where the goal is to connect all vertices with the least total weight, while is an algorithm where the goal is to find the path with the least total weight between two vertices.

sol:

Minimum Spanning Tree (Krusal's / Prim algorithm) Dijkstra's algorithm

25.

In a binary tree, the __ traversal visits the root node before its children, while the __ traversal visits the left subtree, then the root, and finally the right subtree.

sol:

preorder inorder

26.

The postorder traversal and the breadth-first traversal of the following tree are and respectively.

sol:

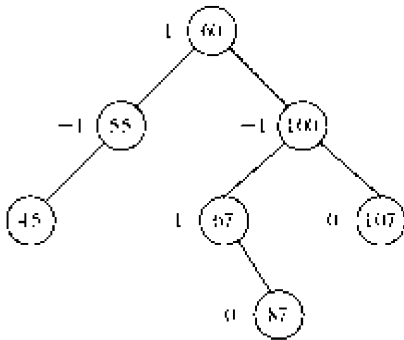
27.

In an AVL tree, a __ imbalance occurs when a node is inserted into the right subtree of the right child of A, while a __ imbalance occurs when a node is inserted into the left subtree of the left child of A.

sol:

RR, LL

28. Given the original AVL tree below, after adding element 81, the __ rotation would be performed to rebalance the tree.



Collisions occur when two different keys produce the same hash value. Besides open addressing, ____, where each hash table slot maintains a linked list of elements mapping to the same slot, is the other way to solve collisions.

sol:

RL

29. Collisions occur when two different keys produce the same __. Besides open addressing, __, where each hash table slot maintains a linked list of elements mapping to the same slot, is the other way to solve collisions.

sol:

- 第一空: hash code (或 index / hash value) —— 两个不同的 key 算出相同的哈希值/索引, 就叫冲突。
- 第二空: separate chaining (分离链表法) —— 每个哈希槽维护一个链表, 映射到同一槽的元素都挂在链表上。这是 open addressing 之外的另一种冲突解决方法。

30. __ hashing uses a secondary hash function h' on the keys to determine the increments, aiming to avoid the problem.

sol:

Double hashing (双重哈希) —— 使用第二个哈希函数 $h'(key)$ 来决定探测步长, 每次不是固定步长 (如 +1), 而是 $(index + i * h'(key)) \% table.length$, 这样可以避免一次聚集 (primary clustering) 的问题。

Hash Table Collision

- A hash table stores data in an array.
- A hash function maps a key to an index.
- Collision (冲突) happens when two different keys are mapped to the same index.

Collision Resolution

- Separate chaining (分离链表法)
 - Each slot stores a linked list.
 - All elements mapped to the same slot are stored in that linked list.

- Open addressing (开放寻址法)
 - All elements are stored inside the hash table array.
 - If a slot is occupied, the algorithm probes another slot.

Double Hashing

- Double hashing is a type of open addressing.
- It uses a second hash function $h'(key)$ to decide the probing step size.
- Formula: $index_i = (h(key) + i * h'(key)) \bmod table.length$
- Purpose: reduce primary clustering (一坨连续位置被占用的问题) .

Part B. Coding Questions

Question 1: You are developing a library management system that categorizes books into three sections: “Fiction,” “Historical,” and “Reference.” Each book is placed in a queue according to its section for shelving.

Given three priority queues representing the books waiting to be shelved: Fiction Queue: { “The Great Gatsby” , “To Kill a Mockingbird” , “1984” , “Pride and Prejudice” , “Harry Potter” , “The Hobbit” } Historical Queue: { “Sapiens” , “Pride and Prejudice” , “A Brief History of Time” , “Educated” } Reference Queue: { “The Great Gatsby” , “A Brief History of Time” , “Harry Potter” , “Encyclopedia Britannica” , “The Hobbit” } You job is to find books that are present in all three sections (i.e. popular books). Complete the 3 tasks indicated in the following whiteboard. Whiteboard (where students code answers):

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;
import java.util.PriorityQueue;
import java.util.Set;

public class Main {
    public static void main(String[] args) {

        // Task 1: Create PriorityQueues for each section of books

        // Task2: Call findPopularBooks method and print out the name of the popular
        // books }
    }

    public static Set<String> findPopularCars(
        PriorityQueue<String> fictionQueue,
        PriorityQueue<String> historicalQueue,
        PriorityQueue<String> referenceQueue) {
        // Task 2: Implement a method called "findPopularCars" that are present in all
        // three sections
    }
}
```

sol:

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;
import java.util.PriorityQueue;
import java.util.Set;

public class Main {
    public static void main(String[] args) {

        // Task 1: Create PriorityQueues for each section of books
        PriorityQueue<String> fictionQueue = new PriorityQueue<>(Arrays.asList(
            "The Great Gatsby", "To Kill a Mockingbird", "1984",
            "Pride and Prejudice", "Harry Potter", "The Hobbit"));
        PriorityQueue<String> historicalQueue = new PriorityQueue<>(
            Arrays.asList("Sapiens", "Pride and Prejudice", "A Brief History of
            ↪ Time", "Educated"));
        PriorityQueue<String> referenceQueue = new PriorityQueue<>(Arrays.asList(
            "The Great Gatsby", "A Brief History of Time", "Harry Potter",
            "Encyclopedia Britannica", "The Hobbit"));

        // Task2: Call findPopularBooks method and print out the name of the popular
        // books }
        Set<String> popularBooks = findPopularCars(fictionQueue, historicalQueue,
            ↪ referenceQueue);
        System.out.println("Popular Books:");
        System.out.println(popularBooks.size());
        popularBooks.stream().forEach(System.out::println);
    }

    public static Set<String> findPopularCars(
        PriorityQueue<String> fictionQueue,
        PriorityQueue<String> historicalQueue,
        PriorityQueue<String> referenceQueue) {
        // Task 2: Implement a method called "findPopularCars" that are present in all
        // three sections
        Set<String> popularBooks = new HashSet<>();

        HashMap<String, ArrayList<Integer>> bookCounts = new HashMap<>();
        for (String book : fictionQueue) {
            bookCounts.putIfAbsent(book, new ArrayList<>());
            bookCounts.get(book).add(1);
        }
        for (String book : historicalQueue) {
            bookCounts.putIfAbsent(book, new ArrayList<>());
            bookCounts.get(book).add(2);
        }
        for (String book : referenceQueue) {
            bookCounts.putIfAbsent(book, new ArrayList<>());
            bookCounts.get(book).add(3);
        }

        for (String book : bookCounts.keySet()) {
            int flag1, flag2, flag3 = 0;
            flag1 = flag2 = flag3 = 0;
        }
    }
}
```

```

        for (int section : bookCounts.get(book)) {
            if (section == 1)
                flag1 = 1;
            if (section == 2)
                flag2 = 1;
            if (section == 3)
                flag3 = 1;
        }

        if (flag1 + flag2 + flag3 == 3) {
            popularBooks.add(book);
        }
    }

    return popularBooks;
}

```

OR

```

import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;
import java.util.PriorityQueue;
import java.util.Set;

public class Main {
    public static void main(String[] args) {

        // Task 1: Create PriorityQueues for each section of books
        PriorityQueue<String> fictionQueue = new PriorityQueue<>(Arrays.asList(
            "The Great Gatsby", "To Kill a Mockingbird", "1984",
            "Pride and Prejudice", "Harry Potter", "The Hobbit"));
        PriorityQueue<String> historicalQueue = new PriorityQueue<>(
            Arrays.asList("Sapiens", "Pride and Prejudice", "A Brief History of
            ↪ Time", "Educated"));
        PriorityQueue<String> referenceQueue = new PriorityQueue<>(Arrays.asList(
            "The Great Gatsby", "A Brief History of Time", "Harry Potter",
            "Encyclopedia Britannica", "The Hobbit"));

        // Task2: Call findPopularBooks method and print out the name of the popular
        // books }
        Set<String> popularBooks = findPopularCars(fictionQueue, historicalQueue,
            ↪ referenceQueue);
        System.out.println("Popular Books:");
        System.out.println(popularBooks.size());
        popularBooks.stream().forEach(System.out::println);
    }

    public static Set<String> findPopularCars(
        PriorityQueue<String> fictionQueue,
        PriorityQueue<String> historicalQueue,
        PriorityQueue<String> referenceQueue) {
        // Task 2: Implement a method called "findPopularCars" that are present in all
    }
}

```

```

// three sections

// Conver the PriorityQueues to Set for easier Comparison
Set<String> fictionSet = new HashSet<>(fictionQueue);
Set<String> historicalSet = new HashSet<>(historicalQueue);
Set<String> referenceSet = new HashSet<>(referenceQueue);

return fictionSet.stream().filter(historicalSet::contains).filter(
    ↪ referenceSet::contains)
    .collect(java.util.stream.Collectors.toSet());
}
}

```

Question 2: A human resource system needs to sort employee records based on different attributes. One common requirement is to sort employees by their ID numbers and another is to sort them by their names in a case-insensitive manner.

Given the following array of employee IDs and names:

```

Integer[] employeeIds = {102, 101, 104, 103, 105};
String[] employeeNames = {"John", "Alice", "bob", "Diana", "Charlie"};

```

```

import java.util.Arrays;
import java.util.Comparator;

public class Main {
    // Task1: Use the following insertionSort() method with the Comparable interface
    // to sort the array of employee IDs in ascending order.

    public static <E extends Comparable<E>> void insertionSort(E[] array) {
    }

    // Task2: Use the following insertionSort() method with the Comparator interface
    // to sort the array of employee names in a case-insensitive manner
    public static <E> void insertionSort(E[] list, Comparator<? super E> comparator)
    ↪ {
    }

    public static void main(String[] args) {
        Integer[] employeeIds = { 102, 101, 104, 103 };
        String[] employeeNames = { "John", "Alice", "Bob", "Charlie" };

        insertionSort(employeeIds);
        System.out.println("Sorted Employee IDs: ");
        System.out.println(Arrays.toString(employeeIds));

        insertionSort(employeeNames, String.CASE_INSENSITIVE_ORDER);
        System.out.println("Sorted Employee Names: ");
        System.out.println(Arrays.toString(employeeNames));
    }
}

```

sol:

```
import java.util.Arrays;
import java.util.Comparator;

public class Main {
    // Task1: Use the following insertionSort() method with the Comparable interface
    // to sort the array of employee IDs in ascending order.

    public static <E extends Comparable<E>> void insertionSort(E[] array) {
        for (int i = 1; i < array.length; i++) {
            E key = array[i];
            int j = i - 1;
            while (j >= 0 && array[j].compareTo(key) > 0) {
                array[j + 1] = array[j];
                j--;
            }
            array[j + 1] = key;
        }
    }

    // Task2: Use the following insertionSort() method with the Comparator interface
    // to sort the array of employee names in a case-insensitive manner
    public static <E> void insertionSort(E[] list, Comparator<? super E> comparator)
    ↪ {
        for (int i = 1; i < list.length; i++) {
            E key = list[i];
            int j = i - 1;
            while (j >= 0 && comparator.compare(list[j], key) > 0) {
                list[j + 1] = list[j];
                j--;
            }
            list[j + 1] = key;
        }
    }

    public static void main(String[] args) {
        Integer[] employeeIds = { 102, 101, 104, 103 };
        String[] employeeNames = { "John", "Alice", "Bob", "Charlie" };

        insertionSort(employeeIds);
        System.out.println("Sorted Employee IDs: ");
        System.out.println(Arrays.toString(employeeIds));

        insertionSort(employeeNames, String.CASE_INSENSITIVE_ORDER);
        System.out.println("Sorted Employee Names: ");
        System.out.println(Arrays.toString(employeeNames));
    }
}
```